

Spectral resolution of 1D QM systems with B-splines

UCF, PHY 5606 B-splines, Fall 2023

Taken by storm. If you need to solve a differential equation numerically, many techniques are available (see W. H. Press, S. A. Teukolsky, W. T. Vetterling, B. P. Flannery *Numerical Recipes*, Cambridge University Press). In the 1990s, however, one tool proved exceptionally accurate for one-dimensional quantum-mechanical problems: the B-spline basis (see the [Wikipedia entry](#) and this [Review on the use of B-splines in AMO Physics](#)). For this reason, several projects in this course use B-splines.

Splines of order k are defined as piecewise polynomials of degree $k - 1$, C^∞ everywhere, except at a non-decreasing set of points called knots $\{t_i, i = 1, 2, \dots, n\}$. This definition gives splines a tremendous amount of flexibility, because they combine the local effectiveness of Taylor expansion without suffering from its weird long-range behavior, while also ensuring a high level of regularity. The splines with support on k consecutive (possibly degenerate) intervals are called B-splines, and they form a basis of the spline space. The i -th B-spline of order k , $B_i^k(x)$, with support $[t_i, t_{i+k}]$, is defined iteratively by de Boor's algorithm as

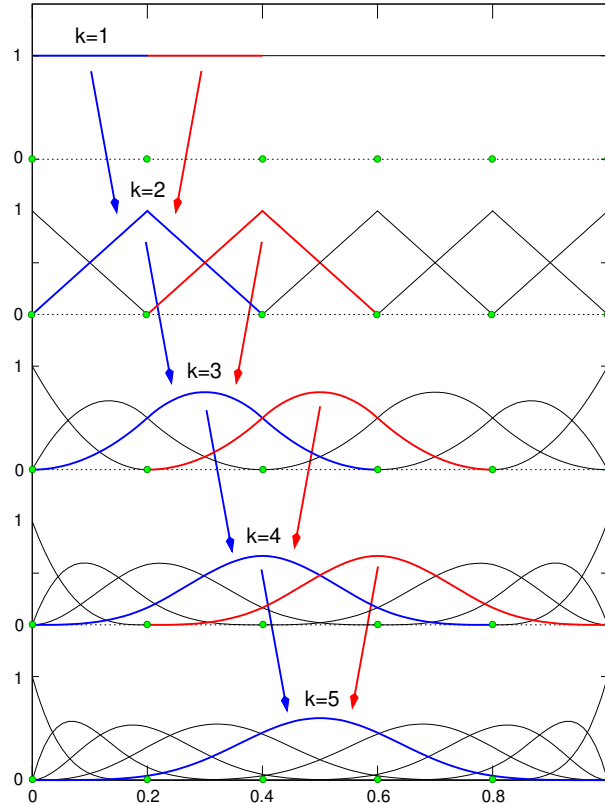
$$B_i^1(x) = \theta(x - t_i) \cdot \theta(t_{i+1} - x)$$

$$B_i^k(x) = \frac{x - t_i}{t_{i+k-1} - t_i} B_i^{k-1}(x) + \frac{t_{i+k} - x}{t_{i+k} - t_{i+1}} B_{i+1}^{k-1}(x). \quad (1)$$

The spline space has dimension $n - k$. If a knot is distinct from its nearest neighbors, then the splines are at least C^{k-2} there. If ν consecutive knots coincide, the regularity of the point decreases, and the splines are at least $C^{k-\nu-1}$ there. It is customary to set the first k knots and the last k knots equal to each other. In this way, the splines can be C^{-1} (i.e., discontinuous but integrable) at the boundaries of the interval $[t_1, t_n]$

$$t_1 = \dots = t_k < t_{k+1} \leq \dots \leq t_{n-k} < t_{n-k+1} = \dots = t_n.$$

With this definition, the B-splines are normalized so that $\sum_i B_i^k(x) = 1$ in the whole interval $[t_1, t_n]$.



B-splines are well suited to represent quantum-mechanical wavefunctions. To obtain bound states of a one-dimensional Hamiltonian, choose a knot sequence dense enough on a domain that contains the state, assemble the Hamiltonian in the B-spline basis with vanishing (Dirichlet) boundary conditions, and diagonalize. For continuum states, boundary conditions and normalization require care, but high accuracy is achievable. For the project, we provide a Fortran 90 B-spline module that evaluates matrix elements of the form

$$\int dx [D^n B_i(x)] f(x; \mathbf{Y}) [D^m B_j(x)]$$

where $B_i(x)$ are B-splines, D^n denotes the n -th derivative, and $f(x; \mathbf{Y})$ is a user-defined real-valued function of a real variable with an optional array of parameters $\mathbf{Y}$. A fortran 2003 template illustrates the module by computing the eigenvalues and eigenfunctions of a particle in a box. For a quick review of the language, see [modern Fortran tutorial](#).

To compile and run the code you need a Fortran 2003 compiler and the [BLAS](#) (Basic Linear Algebra Subprogram) and [LAPACK](#) libraries.

Option 1: Install [intel's oneAPI tools](#), including the compiler suite and MKL. If the installation succeed, compile with

```
ifort -O2 -mkl moduleBspline.f90 Template.f90
```

Option 2: install `gfortran` (on Linux via your package manager; on MacOS via Homebrew). Install BLAS and LAPACK (from your distribution or from [Netlib](#)). After building the libraries, you can compile with:

```
gfortran moduleBspline.f90 Template.f90 -L <LAPACK_INSTALL_DIR> -llapack -lrefblas
```

where `LAPACK_INSTALL_DIR` is the directory containing `liblapack.a` and `librefblas.a`.

In most projects you will only need to adapt the potential, the spatial interval, and the routines that tabulate and write out the results. For time evolution, add the steps required to propagate the expansion coefficients.

Note on the generalized eigenvalue problem. The B-splines $\{|B_i\rangle\}_{i \in I_N}$ are linearly independent but not orthonormal. Suppose we wish to solve the eigenvalue problem

$$\hat{H}|\psi\rangle = |\psi\rangle E$$

. If the B-splines are able to reproduce the wave function, then it is $|\psi\rangle = \sum_j |B_j\rangle c_j$, and the eigenvalue problem becomes

$$\sum_j \hat{H}|B_j\rangle c_j = \sum_j |B_j\rangle c_j E \quad \Rightarrow \quad \sum_j \langle B_i | \hat{H} | B_j \rangle c_j = \sum_j \langle B_i | B_j \rangle c_j E$$

Let's define the Hamiltonian matrix $\mathbf{H}$ as $H_{ij} = \langle B_i | \hat{H} | B_j \rangle$ and the *overlap* matrix $\mathbf{S}$ as $S_{ij} = \langle B_i | B_j \rangle$. Then, the problem becomes

$$\mathbf{H}\mathbf{c} = \mathbf{S}\mathbf{c}E.$$

This is called generalized eigenvalue problem, because it depends on two matrices. There are canned routines that solve this problem efficiently. If one has only a diagonalization routine for the standard problem, however, it is possible to solve the generalized version in two steps. First, diagonalize $\mathbf{S}$,

$$\mathbf{S} = \mathbf{O}\mathbf{\Lambda}\mathbf{O}^t, \quad \mathbf{O}^t\mathbf{O} = \mathbf{1}, \quad \Lambda_{ij} = \lambda_i \delta_{ij}, \quad \lambda_i > 0.$$

Then the generalized problem can be written as

$$\mathbf{H}\mathbf{c} = \mathbf{O}\mathbf{\Lambda}\mathbf{O}^t\mathbf{c}E \quad \Rightarrow \quad \mathbf{\Lambda}^{-1/2}\mathbf{O}^t\mathbf{H}\mathbf{O}\mathbf{\Lambda}^{-1/2}\mathbf{\Lambda}^{1/2}\mathbf{O}^t\mathbf{c} = \mathbf{\Lambda}^{1/2}\mathbf{O}^t\mathbf{c}E$$

It is now sufficient to redefine $\tilde{\mathbf{H}} \equiv \mathbf{\Lambda}^{-1/2}\mathbf{O}^t\mathbf{H}\mathbf{O}\mathbf{\Lambda}^{-1/2}$ and $\mathbf{q} \equiv \mathbf{\Lambda}^{1/2}\mathbf{O}^t\mathbf{c}$, to convert the problem in a traditional eigenvalue problem (notice that the new Hamiltonian is Hermitian, $\tilde{\mathbf{H}}^\dagger = \tilde{\mathbf{H}}$)

$$\tilde{\mathbf{H}}\mathbf{q} = \mathbf{q}E \quad \text{after solving it, the original coefficients are} \quad \mathbf{c} = \mathbf{O}\mathbf{\Lambda}^{-1/2}\mathbf{q}$$

```

1  !> Computes the eigenvalues of a number of states of the particle in the box \[0,a]\$
2  !! and compares them with the analytically known result, \[E_n = (\pi^2 n^2/a^2)/2\$
3  !! where \[hbar=1\$ and \[m=1\$
4  program main
5
6  use, intrinsic :: iso_fortran_env, only : ERROR_UNIT, OUTPUT_UNIT
7  use ModuleBSpline
8
9  implicit none
10
11  !.. Bspline parameters and variables
12  integer          , parameter      :: BS_NNODS = 51
13  integer          , parameter      :: BS_ORDER = 12
14  real(kind(1d0)), parameter      :: BS_GRMIN = 0
15  real(kind(1d0)), parameter      :: BS_GRMAX = 1.d0
16  real(kind(1d0)), parameter      :: BS_INTER = BS_GRMAX - BS_GRMIN
17  real(kind(1d0))  , parameter      :: BS_GRID(BS_NNODS)
18  type(ClassBSpline)      :: BSpline
19
20  !.. Hamiltonian, overlap, and spectral-decomposition arrays
21  real(kind(1d0)), allocatable :: Hmat(:,,:), Smat(:,,:)
22  integer          :: iNode, nEn, info
23
24  !.. Initializes the BSpline set
25  do iNode=1,BS_NNODS
26      BS_GRID(iNode) = BS_GRMIN + &
27          BS_INTER * dble(iNode-1) / dble(BS_NNODS-1)
28  enddo
29  call BSpline%Init( &
30      BS_NNODS      , &
31      BS_ORDER      , &
32      BS_GRID       , &
33      info          )
34  if(info/=0) error stop
35
36  !.. Due to the continuity requirement on the wavefunction,
37  !   the first and last BSplines, which do not vanish at the
38  !   boundary of the interval, cannot be used
39  nEn = BSpline%GetNBsplines() - 2
40
41  call FillMatrices()
42
43  call GeneralizedEigenproblem()
44
45  contains
46
47  subroutine FillMatrices()
48      integer          :: iBs1, iBs2
49      real(kind(1d0)) :: Overlap, KineticEnergy, PotentialEnergy
50      real(kind(1d0)) :: parvec(1)
51      procedure(D2DFUN), pointer :: fPtrUni, fPtrPot
52      fPtrUni => Unity
53      fPtrPot => Potential
54      allocate(Smat(BS_ORDER,nEn))
55      Smat=0.d0
56      allocate(Hmat,source=Smat)
57      parvec(1)=0.d0
58      do iBs2 = 2, nEn + 1
59          do iBs1 = max(2, iBs2-BE_ORDER+1), iBs2
60              Overlap = BSpline%Integral(fPtrUni,iBs1,iBs2)
61              KineticEnergy = BSpline%Integral(fPtrUni,iBs1,iBs2,1,1) / 2.d0
62              PotentialEnergy = BSpline%Integral(fPtrPot,iBs1,iBs2,parvec=parvec)
63              Smat(iBs1+BS_ORDER-iBs2,iBs2-1) = Overlap
64              Hmat(iBs1+BS_ORDER-iBs2,iBs2-1) = KineticEnergy + PotentialEnergy
65          enddo
66      enddo
67  end subroutine FillMatrices
68

```

```

69
70 subroutine GeneralizedEigenproblem()
71   real(kind(1d0)), allocatable :: Eval(:), Evec(:, :), Work(:)
72   real(kind(1d0)), parameter   :: ERROR_THRESHOLD = 1.d-10
73   real(kind(1d0)), parameter   :: PI = acos(-1.d0)
74   integer, parameter           :: npts=301
75   real(kind(1d0))              :: EigenvalueError, x, fun
76   integer                      :: iEn, ix, info, uid
77   character(len=10)            :: iEnStrn
78
79   !.. Solve the generalized eigenvalue problem for banded matrices
80   allocate(work(3*nEn), Eval(nEn), Evec(nEn+2, nEn))
81   work=0.d0
82   Eval=0.d0
83   Evec=0.d0
84   call DSBGV( 'V', 'U', nEn, BS_ORDER-1, BS_ORDER-1, &
85             Hmat, BS_ORDER, Smat, BS_ORDER, Eval, Evec(2,1), nEn+2, work, info )
86   if( info /= 0 )then
87     write(ERROR_UNIT,"(a,i0)") "DSBGV info : ", info
88     error stop
89   endif
90   deallocate(work)
91
92   do iEn = 1, nEn
93
94     !.. Print the eigenvalue on screen
95     EigenvalueError = Eval(iEn) - (iEn*PI)**2/2.d0
96     if( EigenvalueError > ERROR_THRESHOLD )exit
97     write(OUTPUT_UNIT,"(i4,*(x,e24.16))" iEn, Eval(iEn), EigenvalueError
98
99     !.. Print the eigenfunction on file
100    write(iEnStrn,"(i000.3)") iEn
101    open(newunit = uid, &
102         file      ="EigenState_"//trim(adjustl(iEnStrn)), &
103         form      ="formatted", &
104         status     ="unknown")
105    do ix = 1, npts
106      x = BS_GRMIN + (BS_GRMAX - BS_GRMIN)/(npts-1)*(ix-1)
107      fun = BSpline%Eval(x, Evec(:, iEn))
108      write(uid,"(*(x,e24.16))" x, fun
109    enddo
110    close(uid)
111
112  enddo
113  write(OUTPUT_UNIT,"(a,i0)") "Number of Accurate Eigenvalues : ", iEn-1
114 end subroutine GeneralizedEigenproblem
115
116 Pure real(kind(1d0)) function Unity(x, parvec) result(y)
117   DoublePrecision, intent(in) :: x
118   DoublePrecision, optional, intent(in) :: parvec(*)
119   y=1.d0
120 end function Unity
121
122 Pure real(kind(1d0)) function Potential(x, parvec) result(res)
123   real(kind(1d0)), intent(in) :: x
124   real(kind(1d0)), optional, intent(in) :: parvec(*)
125   res = 0.d0
126 end function Potential
127
128 end program main

```